UNIVERSITÉ Sultan Moulay Slimane

Faculté des Sciences / Département d'Informatique

Filière / Master : M2 AIDC



TP 4 : RAG

Implémentation d'un système RAG (Retrieval-Augmented Generation)

Réalisé par :

AHLAME LAOUAD

Encadré par :

Pr. Lhoussaine Daoudi

Année Universitaire : 2026-2027

Date : Octobre 2026

1 Introduction

Dans le domaine du Traitement Automatique du Langage Naturel (NLP), les grands modèles de langage (Large Language Models – LLMs) démontrent des capacités avancées de raisonnement et de génération textuelle. Toutefois, ils présentent deux contraintes majeures : une limite temporelle dans leurs connaissances intrinsèques et une propension aux hallucinations factuelles lorsqu'ils sont interrogés sur des données spécifiques ou privées.

Pour pallier ces faiblesses sans recourir à un réentraînement ou à un ajustement fin (fine-tuning) coûteux en ressources, l'architecture RAG (Retrieval-Augmented Generation) s'impose aujourd'hui comme une solution standard. Le principe consiste à combiner un système de recherche documentaire à base vectorielle (Retrieval) et un modèle génératif (Generation) chargé de formuler une réponse fondée exclusivement sur les extraits contextuels pertinents récupérés.

Ce travail pratique (TP) a pour objectif la conception, l'implémentation et l'évaluation critique d'un pipeline RAG complet développé avec l'écosystème LangChain, une base vectorielle ChromaDB et des modèles de langage avancés (Groq / Google Gemini).

2 Qu'est ce que le RAG ?

Le déroulement de ce projet RAG s'articule autour de quatre axes fondamentaux :

- **L'ingestion et la segmentation (Chunking) :**le chargement d'un document PDF spécifique (CV au format ATS) et son découpage optimisé en segments via un algorithme de fenêtrage de tokens (RecursiveCharacterTextSplitter avec tiktoken).
- **L'indexation vectorielle :**la transformation des segments textuels en représentations denses à l'aide d'un modèle d'embedding (Sentence-Transformers / Hugging Face) et leur stockage persistant dans ChromaDB.
- **L'interrogation et le prompt engineering :**l'extraction par similarité sémantique (similarity search) des k-meilleurs fragments et l'injection contrôlée de ce contexte dans un gabarit de prompt strict visant à contraindre le comportement du modèle et à supprimer les extrapolations.
- **L'évaluation par LLM (LLM-as-a-Judge) :**la mise en œuvre d'un protocole d'arbitrage automatisé évaluant formellement l'ancrage factuel (groundedness / faithfulness) des réponses produites, selon une échelle de conformité graduée et une démarche de raisonnement pas-à-pas (Chain-of-Thought).

3 Réalisation TP :

3.1 La préparation de l'environnement :

Avant de commencer la manipulation des différentes tâches en doit met en place un environnement bien organisé et fiable correctement :

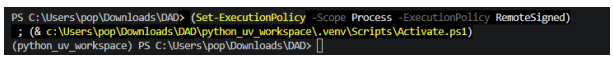


FIGURE 1 – Activation de l'environnement

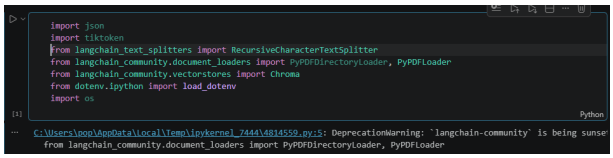


FIGURE 2 – Les importations nécessaires

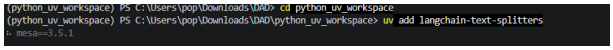


FIGURE 3 – Ajout des splitters

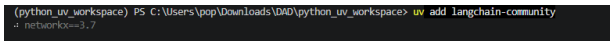


FIGURE 4 – Ajout de la communaté langchain



FIGURE 5 – imporation des clé api du fichier .env



FIGURE 6 – Appel du modèle LLM

3.2 Le téléchargement du pdf à manipuler chunking :

Pour tester l'architecture RAG et ses différentes parties on a utilisé notre propre CV pour poser les différentes questions et évaluer vers la fin la qualité des réponse en utilisant un autre LLM avec d'être un juge.



FIGURE 7 – Téléchargement du cv personnel

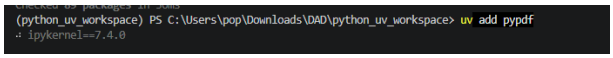


FIGURE 8 – Ajout de pypdf



FIGURE 9 – Lecture du pdf avec pypdfLoader



FIGURE 10 – Splitting du text pdf

La figure ci-dessus affiche l'opération d'utilisation d'encodeur pour transformer la totalité du document en blocs.

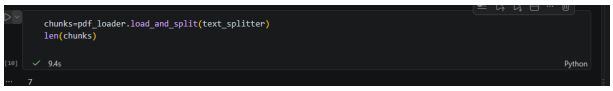


FIGURE 11 – Le nombre de blocs dans le documents inséré

3.3 vectore Store - ChromaDB et Embedding

Après cette étape en doit chisir un autre modèle pour faire la vectorisation de nos blocs en nombres à stocker dans dans notre database afin de faire les test de similarité lors de la recherche des réponses pour les questions utilisateur :



FIGURE 12 – installation de la bibliothèque modèle

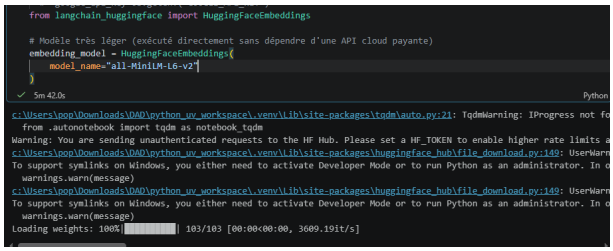


FIGURE 13 – Appel du modèle

Pour stocker les informations extraites, on a crée un dossier nommé "store" chromadb pour les garder.

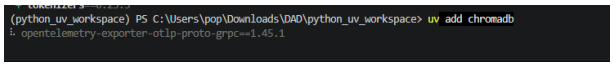


FIGURE 14 – Installation chromadb



FIGURE 15 – Stockage des blocs du documents

3.4 Trouvaille des chunks liée à la questions (Retriever) :



FIGURE 16 – transformation du vectorstore en un composant de recherche documentaire

Pour le test on va essayer de poser une question en appelant le retrieval pour un test de similarité ainsi le nombre de blocs :

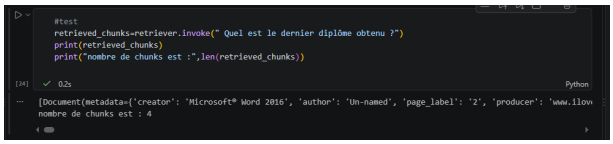


FIGURE 17 – nombre de blocs

3.5 Prompt Design :

Dans cette partie on transmettera à notre llm la prompt dont il doit se baser pour générer sa réponse ainsi éviter l'hallucination lors de la génération des réponses.

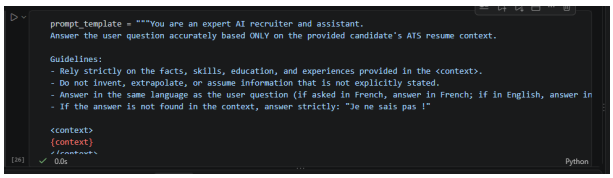


FIGURE 18 – La prompt

3.6 Retrieving the Relevant Documents :

Dans cette partie nous allon sfaire un test de la capacité de notre LLM dans la trouvaille d'une réponse correcte et dans le contexte demandé .

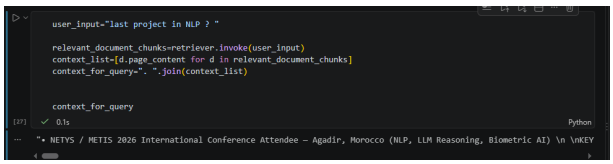


FIGURE 19 – évaluation de LLM dans la génération de réponses



FIGURE 20 – La taille des chuncks les plus pertinetes

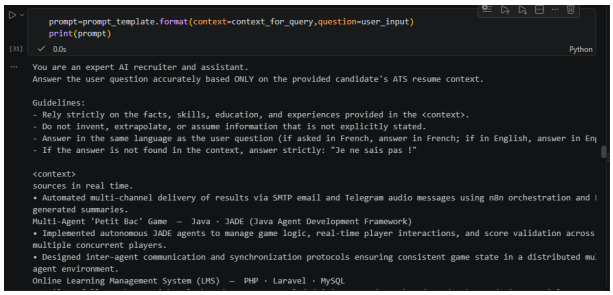


FIGURE 21 – Injection du contexte das la prompt

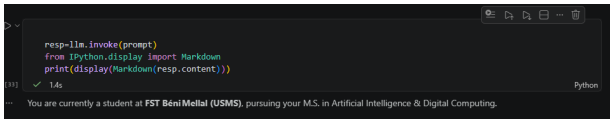


FIGURE 22 – Utilisation du prompt pour généréer la réponse

3.7 Le prompt :

3.8 Récupérer la réponse de la part de llm :

3.9 Regroupement dans une application RAG :

Ici, on va appliquer le processus complet des RAG en commençant par la définition de la fonction :



FIGURE 23 – fonction d'orchestration l'ensemble de la chaîne Retrieval-Augmented Generation

Maintenat appelons le RAG :



FIGURE 24 – La réponse dans l'état de l'information existe

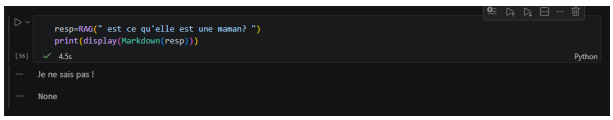


FIGURE 25 – La réponse dans l'état ou il ne sait pas ou il ne trouve pas l'information

3.10 Évaluation de la réponse du LLM :

Pour construire l'architecture et le contexte on a suivi les étapes suivantes :



FIGURE 26 – La question à poser



FIGURE 27 – Récupération contexte

3.11 Groundness :

Dans cette section, on va tester la fialbilité des réponses en utilisant un autre llm pour juger le niveau de conservation et fiabilité de réponse :

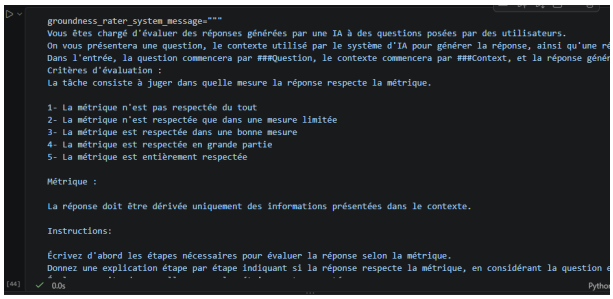


FIGURE 29 – Prompt pour le juge



FIGURE 30 – Appel du modèle juge

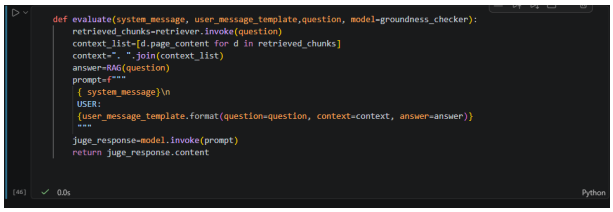


FIGURE 31 – Définition de la fonction d'évaluation



FIGURE 28 – récupération réponse



FIGURE 32 – Appel de la fonction et affichage de la logique d'évaluation

Protocole d'Évaluation LLM-as-a-Judge : Métrique d'Ancrage Factuel

Étapes pour évaluer la conformité à la métrique :
« La réponse doit être dérivée uniquement des informations présentées dans le contexte. »

1. **Identifier la question**
   - Lire la partie commençant par `###Question` afin de comprendre ce qui est demandé.
   - **Requête analysée :** *« Quels sont les domaines d'intérêts de Ahlame Laouad ? »*
2. **Analyser le contexte**
   - Parcourir le texte sous `###Context` et relever toutes les informations qui décrivent les compétences, les projets, les expériences et les domaines d'expertise de Ahlame Laouad.
   - Repérer les termes clés (ex. : IA, Machine Learning, NLP, systèmes agentiques, MLOps, vision par ordinateur, développement full-stack, marketing digital).
3. **Comparer le contenu de la réponse avec le contexte**
   - Vérifier que chaque domaine mentionné dans la réponse apparaît explicitement ou implicitement dans le contexte extrait.
   - S'assurer qu'aucune information supplémentaire (non-présente dans le contexte) n'a été ajoutée.
4. **Détecter les éventuelles extrapolations ou inventions**
   - Rechercher des éléments qui ne sont pas soutenus par le texte du contexte (par exemple, des intérêts personnels, des loisirs ou des technologies non citées).
   - Si de tels éléments existent, la métrique n'est pas respectée.
5. **Évaluer le degré de conformité**
   - Si toutes les affirmations proviennent strictement du contexte sans ajout externe, la métrique est entièrement respectée (**score 5**).
   - Si la réponse omet des données ou ajoute du contenu non vérifiable, le score diminue proportionnellement (**4** $\rightarrow$ **1**).

Application pratique des étapes à l'exemple fourni :

1. **Question :** Quels sont les domaines d'intérêts de Ahlame Laouad ?
2. **Informations pertinentes extraites du contexte :**
   - Intelligence Artificielle & Machine Learning (Deep Learning, Transformers, BERT, LLMs).
   - Traitement du Langage Naturel (NLP, analyse de sentiments, DarijaBERT, tokenisation, classification de séquences).
   - Systèmes d'IA Agentiques & Multi-agents (JADE, LangGraph, LangChain, orchestration).
   - MLOps & Déploiement (Docker, FastAPI, API REST, pipelines de production).
   - Ingénierie des Données (pré-traitement, feature engineering, bases MongoDB, MySQL, SQLite).
   - Vision par Ordinateur (NASNet-Large).
   - Développement Full-Stack (Laravel, HTML5, CSS3, JavaScript, PHP).

1

– Gestion de projets numériques & Marketing digital (leadership d'équipes à distance, campagnes en ligne).

3. **Correspondance avec la réponse générée :** La réponse synthétise fidèlement ces domaines d'expertise à partir des termes et concepts exacts mentionnés dans le CV ATS. Aucun domaine non documenté n'a été inséré.
4. **Extrapolation ou invention :** Aucune hallucination constatée. L'intégralité du contenu est traçable dans les segments textuels du document ingéré.
5. **Évaluation globale :** La réponse est rigoureusement ancrée dans le contexte fourni.

Score attribué : 5 / 5 – *La métrique est entièrement respectée (fidélité maximale sans hallucination).*

Échelle de Notation pour la Fidélité au Contexte (*Faithfulness*)

Score	Signification	Description
5 / 5	**Excellent (Parfait)**	La réponse utilise **uniquement** les informations du contexte. Aucune hallucination, aucune extrapolation.
4 / 5	**Bon**	La réponse est correcte et s'appuie sur le contexte, mais la formulation pourrait être légèrement plus précise.
3 / 5	**Moyen**	La réponse est partiellement vraie, mais elle omet un détail important du contexte ou manque de clarté.
2 / 5	**Médiocre**	La réponse contient des extrapolations ou des informations non vérifiables avec le contexte fourni.
1 / 5	**Mauvais (Hallucination)**	La réponse est totalement hors-sujet, fausse, ou invente des données qui ne sont pas du tout dans le contexte.

2

4 Conclusion :

Ce travail pratique a permis de concevoir, d'implémenter et d'évaluer un pipeline complet de Génération Augmentée par Récupération (RAG) appliqué à l'interrogation d'un document semi-structuré (CV au format ATS). L'ensemble de la chaîne de traitement a été validé à chaque étape clé :

– **Indexation et segmentation :**Le découpage par fenêtrage de tokens (RecursiveCharacterTextSplitter avec tiktoken) couplé à un modèle d'embedding dense a permis d'isoler ef-

ficacement les compétences et les expériences professionnelles sans rupture contextuelle majeure.

- **Recherche sémantique :** L'indexation sous ChromaDB et la configuration du retriever par similarité cosinus ($k = 5$) garantissent l'extraction ciblée des fragments les plus pertinents par rapport à la requête.
- **Contrôle des hallucinations :** L'ingénierie du prompt système (System Prompting), intégrant des délimiteurs stricts et une consigne de repli explicite (« Je ne sais pas ! »), a démontré son efficacité pour contraindre le modèle de génération aux seuls faits énoncés.
- **Évaluation automatisée (LLM-as-a-Judge)** La mise en place d'un protocole d'arbitrage pas-à-pas (Chain-of-Thought) a confirmé la fidélité stricte (groundedness / faithfulness) des réponses générées avec l'obtention du score maximal (5/5), attestant de l'absence totale d'extrapolations. En perspective, ce système pourrait être enrichi par l'intégration d'un mécanisme de re-classement sémantique (Re-ranking via un Cross-Encoder) afin d'optimiser l'ordonnancement des fragments extraits, ainsi que par l'ajout de filtres sur métadonnées pour affiner les requêtes temporelles ou thématiques